

RESEARCH ENGINEER TAKE-HOME / TWO DAYS

Predict Frontier Difficulty

Latent reasoning portfolio

The assignment

Build five agent tasks in Harbor format. Forecast which tasks will be hard before frontier evaluation, then revise the tasks and forecasts using proxy results.

A successful submission includes at least one **valid, hard task** and defensible forecasts. A hard task must have a passing canonical solution, a fair verifier, meaningful agent work, and **at most two target-model passes in eight trials**. Infrastructure and harness failures do not count as model failures.

Model restriction

Never run Fable or GPT-5.6 Sol, including for authoring or critique. Other models, including Claude Opus/Sonnet and open models, may be used for authoring, red-teaming, and pilots.

Shared requirements

Use tiny models with $d_{\text{model}} = 32\text{--}64$, one or two layers, and short sequences. Use float64, fixed seeds, deterministic algorithms, and one CPU thread. Tests must finish in seconds on CPU. Bake all dependencies into the environment; verification must not require network access. Keep a private reference implementation in `/tests`. Every hidden check must follow an explicit instruction.

QUESTION 1 / batched-halting

Implement correct batched inference

Given: A looped model with shared weights, a sigmoid halting head, correct `infer_single(x, mask)`, and an incomplete or naive `infer_batched(X, mask)`. Visible tests use uniform-depth, unpadded batches.

Task: Implement `infer_batched` so that each sample matches isolated `infer_single` execution.

- Match outputs, loop counts N_i , remainders R_i , and ponder cost within absolute tolerance 10^{-10} in float64.
- Stop at cumulative halting probability $\geq 1 - \epsilon$, or at `max_loops`. Specify the exact weighted-output formula, with the remainder on the final step.
- Keep halted states and accumulators unchanged in later iterations. Padding must not affect non-padding outputs.
- Do not modify `infer_single`, model weights, or architecture.

Required coverage: Mixed halting depths, immediate halting, maximum-loop termination, batch size one, large batches, padded variable lengths, and new seeds and shapes. Accept materially equivalent correct implementations.

QUESTION 2 / tbptt-gradient-bug**Repair truncated backpropagation**

Given: A looped model and `train.py` for a toy task requiring several loops. The truncated-backpropagation window is W , but the current detach placement allows gradients through only the last loop. The reported symptom is that accuracy does not improve when inference loops increase.

Task: Fix the training code to implement the intended truncated BPTT semantics.

- In an unroll of L loops, gradients must flow through the last W loops; states from earlier loops must be detached. State the precise definition.
- Do not change model architecture, data, loss, or optimizer hyperparameters.
- Parameter gradients must match a float64 reference on fixed batches for $W \in \{1, 2, 4\}$ and $L \in \{4, 8\}$, within a tight tolerance.
- In a short seeded training run of under 60 CPU seconds, accuracy at $L = 8$ must exceed accuracy at $L = 2$ by the specified margin.

Restrictions: Increasing the learning rate, training longer, removing all detaches, or changing loop counts or architecture does not satisfy the task.

QUESTION 3 / latent-kvcache**Repair cached latent-step generation**

Given: A tiny decoder language model supporting `inputs_embeds`, a correct `generate_full` with arguments `prompt`, `n_latent`, and `n_tokens`, and a buggy `generate_cached`. The full version recomputes the sequence at each step.

During latent steps, the final hidden state **after the final layer normalization and before the language-model head** becomes the next input embedding. The model then decodes `n_tokens` greedy tokens.

Task: Make cached generation match full generation.

- Return identical tokens and matching logits within absolute tolerance 10^{-9} for all supported inputs.
- Define the feedback state precisely. Latent steps must occupy positions, advance position IDs, and be included in the cache.
- Support left-padded batches of different prompt lengths. State the attention-mask and position-ID conventions exactly.
- Cover $n_{\text{latent}} \in \{0, 1, 5\}$, long prompts, mixed prompt lengths, and $n_{\text{tokens}} > 1$.

Specification decision: State whether full recomputation is allowed. If KV-cache use is compulsory, make that requirement explicit and verify it fairly. Output-equivalent full recomputation must otherwise be accepted.

QUESTION 4 / grpo-length-bias

Correct the training objective

Given: A tiny categorical policy over sequences, a small vocabulary with EOS, and a reward independent of response length. The GRPO trainer divides loss by each sequence’s length and normalizes advantages by the group’s reward standard deviation. Response length grows without reward improvement.

Task: Implement the intended objective exactly.

- Compute advantage as reward minus group mean, with no standard-deviation division.
- Sum over response tokens and divide by the constant $\text{max_len} \times \text{group_size}$; do not divide by each response’s own length.
- Preserve the given clipping and KL terms, and specify them explicitly.
- Do not change the reward function.

Required coverage: Loss and gradients must match the reference on fixed batches. In a short seeded run, mean response length must remain within the specified band.

Restrictions: Do not add a reward length penalty, fix only advantage normalization, shorten `max_len`, or change sampling.

QUESTION 5 / act-from-spec

Implement Adaptive Computation Time

Given: A looped cell with an incomplete `act_forward()` function.

Task: Implement ACT from a complete written specification.

- Define per-step halting probabilities and threshold $1 - \epsilon$.
- Define the halting step $N(t)$ and remainder

$$R(t) = 1 - \sum_{\text{steps before halting}} p.$$

- Define the weighted output and ponder cost $N + R$.
- State exact formulas and handle halting at the first step and reaching the maximum step count.
- Match reference outputs, N , R , ponder cost, and gradients across seeds and edge cases.

Restrictions: The verifier must reject incorrect step counts, a remainder applied to the wrong step, ponder cost missing R , and missing gradients through halting probabilities.

Validation required for all five tasks

Run the oracle five times and require reward one every time. Require each listed incorrect shortcut to receive reward zero and a materially different correct solution to receive reward one. Check that the starter builds, visible tests run, verification stays below its timeout, and tasks require meaningful work. Keep shortcut implementations private and record their rewards.

SUBMISSION REQUIREMENTS

Forecast, evaluate, and report

1. Package each task in Harbor format

Include `task.toml`, `instruction.md`, `environment/Dockerfile`, `environment/repo/`, `tests/test.sh`, private tests and reference code, `solution/solve.sh`, and `solution/solution.patch`. Hidden tests run after the agent finishes; the solution is available only to the oracle. Write binary reward one or zero to `/logs/verifier/reward.txt`.

2. Day 1: pilots and preregistered forecasts

Run three or four trials per task using strong, permitted non-target models. Read every trajectory, record pass counts and step-indexed failures, and harden tasks that pilots solve easily.

Submit `day1.md` with task rank, capability gap, concrete failure mechanism, predicted $K/8$, evidence, and prompts/agents/tools/orchestration. Include a short methods paragraph explaining task selection, Task 5's calibration role, and forecasting. Forecasts must be registered before the results they predict are known.

3. Implement `forecast.py`

For each task, use 100,000 Monte Carlo samples:

$$\begin{aligned} p_{\text{proxy}} &\sim \text{Beta}(1 + k, 1 + n - k), \\ \text{logit}(p_{\text{front}}) &= \text{logit}(p_{\text{proxy}}) + \delta, \\ \delta &\sim \text{Normal}(\mu, \sigma), \quad K \sim \text{Binomial}(8, p_{\text{front}}). \end{aligned}$$

Use Day 1 pilot evidence or Day 2 proxy evidence for k/n . Choose μ by failure type: approximately +1.5 to +2.5 for capability failures, or +0.5 to +1.0 for discipline failures. Use a wide σ , approximately one. Output $P(K = 0), \dots, P(K = 8)$ and $P(K \leq 2)$. Avoid overconfident forecasts.

4. Day 2: classify results and revise

Label every trial **INFRA**, **UNFAIR**, **SHALLOW**, **MECHANISM**, or **PASS**. Fix unfair verification first. Keep promising tasks with zero or one pass and substantive mechanism failures; clarify setup when failures are mostly shallow. Harden or retire tasks with three or more passes. Revalidate revised packages and focus remaining effort on the best one or two tasks.

5. Submit a one-page report.md

Include changes and their motivating trials; pass rates k/n with intervals and small-sample limitations; step-indexed trajectory evidence; all five $P(K = 0), \dots, P(K = 8)$ distributions; the hardest task and a falsifier; and the division of generation, implementation, critique, testing, and revision across agents or roles.

Final deliverables

Five task packages, `day1.md`, Day 2 revisions, `report.md`, `forecast.py`, and an `evidence/` folder containing shortcut results, pilot logs, and trajectory labels.